# Architecture Documentation for an Enterprise RAG System

## 1. Introduction

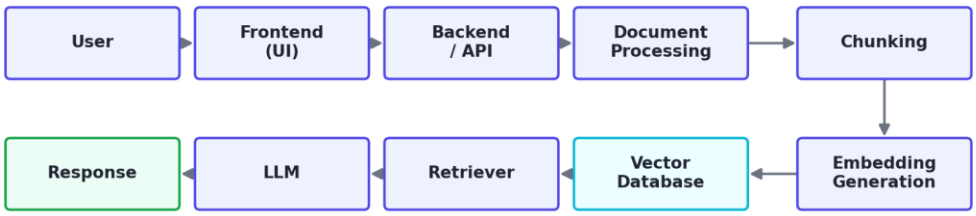
An RAG system is an AI setup that lets a company's staff or customers ask questions and get answers based on the company's own documents, not just general internet knowledge. RAG stands for Retrieval-Augmented Generation. It works by first searching company files for relevant information, then giving that information to a language model to write the final answer.

Architecture documentation explains how all the parts of a system fit together and communicate with each other. It is important because it helps developers understand the system quickly, makes it easier to fix problems, and helps new team members get up to speed without guessing how things work. The purpose of this document is to describe each component of an Enterprise RAG system, show how data moves through it, and explain why the design works the way it does.

## 2. Overall Enterprise RAG Architecture

The diagram below shows the full system from start to finish. A user sends a question through the frontend. The backend passes it to the retrieval and generation pipeline, which searches stored knowledge and builds a final answer using the language model.

**Figure 1: Complete Enterprise RAG Architecture**




*The full pipeline, from the user's question to the final generated response.*

The flow works like this: the user asks something through the Frontend. The Backend/API receives it and manages the whole request. Document Processing and Chunking prepare uploaded files ahead of time, turning them into small pieces of text. The Embedding Generation step turns those pieces into vectors, which are stored in the Vector Database. When a question comes in, the Retriever searches the Vector Database for the closest matching chunks, sends them to the LLM, and the LLM writes the Response that goes back to the user.

## 3. Component Documentation

The table below summarizes every component: what it does, what goes in and out, and why it matters. Components are listed in the order data flows through them.

| Component | Purpose | Input | Output | Why It Matters |
|---|---|---|---|---|
| **User** | Anyone asking a question. | A typed question | Question sent onward | Every request starts here. |
| **Frontend (UI)** | The screen users interact with. | Typed text | Formatted request | The only part users actually see. |
| **Backend / API** | Manages requests, connects all parts. | Frontend request | Pipeline instructions | Keeps the system organized and secure. |
| **Document Processing** | Reads files and extracts clean text. | PDF, DOCX, TXT, CSV | Clean text | Messy files would confuse later steps. |
| **Chunking** | Splits text into small, overlapping pieces. | Clean text | Text chunks | Models work better with short, focused text. |
| **Embedding Generation** | Turns each chunk into a vector. | Text chunks | Vectors | Lets the system search by meaning, not keywords. |
| **Vector Database** | Stores vectors, searches them fast. | Vectors | Matching chunks | Fast search matters once there are many documents. |

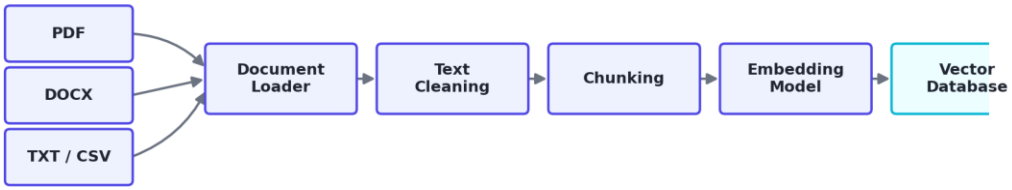
| Component | Purpose | Input | Output | Why It Matters |
|---|---|---|---|---|
| **Retriever** | Finds chunks relevant to a question. | Question vector | Top-k chunks | Good retrieval makes the difference in answer quality. |
| **LLM** | Writes an answer using retrieved chunks. | Question + chunks | Written answer | Turns raw information into a clear reply. |
| **Final Response** | Delivers the answer to the user. | LLM's answer | On-screen message | This is where the user sees the value. |

Each component passes its output to the next one in the list above, from User down to Final Response, which is why keeping this order clear in documentation matters: a change in one component's output format can break the next component's input.

# 4. Additional Architecture Diagrams

The diagrams below zoom into specific parts of the system, showing how documents are ingested, how a question gets answered, how chunking works, how embeddings are stored, and how retrieval produces a final response.

**Figure 2: Document Ingestion Pipeline**



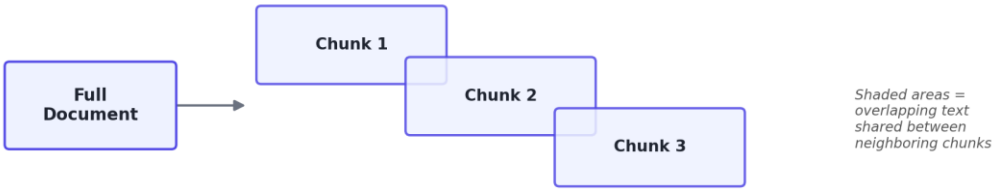

*Uploaded files are loaded, cleaned, split into chunks, embedded, and stored.*

**Figure 3: User Query Processing**




*A question moves through the backend, gets embedded, and is matched against stored chunks.*

**Figure 4: Chunking Process**



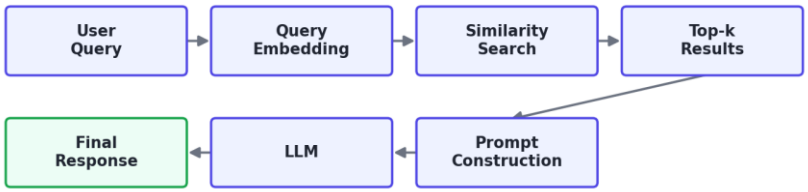

*One document is split into overlapping chunks so information at the edges is not lost.*

**Figure 5: Embedding and Vector Storage**




*Each chunk becomes a vector of numbers, which is then stored in the vector database.*

**Figure 6: Retrieval and Response Generation**




*The system searches for similar chunks, builds a prompt, and the LLM generates the final response.*

# 5. Data Flow

There are two main flows in this system: setting up the knowledge base, and answering a question.

- Setup flow: the user uploads documents -> the documents are processed -> chunks are created -> embeddings are generated -> the embeddings are stored in the Vector Database.
- Question flow: the user asks a question -> the Retriever searches for similar chunks -> the LLM receives the question plus the matching chunks as context -> the final response is generated and sent back to the user.

# 6. Conclusion

Every part of an Enterprise RAG system has a clear, specific job, and the whole design only works well when these parts pass information to each other smoothly. Good architecture improves performance, because each stage is optimized for one task instead of doing everything at once. It improves scalability, since parts like the Vector Database can grow independently as more documents are added. It improves accuracy, because careful retrieval keeps the LLM grounded in real company data instead of guesses. And it improves maintainability, since a clearly documented, modular system is much easier to update, debug, and extend over time.